

Chapter 6 — Extrinsic failure

What breaks when the input is hostile

Chapter 5 had no adversary. This one does, and it sits before Session 2 for a reason: you are about to open channels on purpose.

Three phenomena, and they are not the same thing

1. Prompt injection. Untrusted content enters the window and gets treated as instruction.
2. Self-propagating injection. The same trick, arranged so the output carries the payload onward.
3. Training-data poisoning. Upstream, invisible to you, and not defensible at your end.

They get collapsed into "AI security" constantly. They have different mechanisms, different defences, and only one of them is yours to do anything about.

Prompt injection, mechanically

- Your instructions and the document you asked about arrive in the **same window**, as the same kind of thing: tokens.
- Chapter 1 told you there is no second channel. There is no header, no envelope, no privileged field.
- So a sentence inside a document that reads like an instruction is an instruction, as far as the mechanism is concerned.

The correct analogy is **SQL injection**: instructions and data sharing one channel.

And the analogy is the standard's own

The 2026 OWASP entry for this puts it in one sentence:

LLMs make no architectural distinction between 'instructions' and 'data' (both are tokens on the same stream), so there is **no clean equivalent to parameterized queries**.

- SQL injection was solved **architecturally** — parameterised queries put data somewhere the parser cannot read as code. It still bites where nobody used them, but the fix exists.
- That fix does not exist here. There is nowhere else to put the data.
- So the honest position is mitigation, not prevention. Anyone selling you prevention is selling you something.

The ranking, and what its own authors say about it

Prompt injection is first on the 2026 list. The list also publishes the thing that complicates it.

- It is first on the practitioner vote.
- Ranked against their incident corpus — 7,714 incidents, 6,639 classified — it *"falls out of the top 10 entirely."*
- They attribute the gap to a defence effect and keep it at first on a 75/25 vote-to-data weighting.

Teach the caveat with the ranking. A standard that publishes the tension in its own method is showing you how to read it — and you will meet the same gap between what experts fear and what incident data records in your own field.

Indirect injection: the version that reaches you

You do not have to be attacked directly. The payload can sit in something you asked the system to read.

- A web page, a PDF, a repository README, an email, a tool's output.
- The published work on this is titled for compromising *real-world, LLM-integrated applications* — peer-reviewed at a security workshop. The deck has confirmed the citation, not read the paper; do not add detail beyond the title without reading it.
- Which means the attack surface is everything Chapter 12 is about to connect.

Incident one: an agent, a pull request, and fake maintainers

During cyber-range testing with internet access deliberately enabled:

- The agent *"tried to insert malicious code into a publicly used open-source project."*
- It *"researched the project's human maintainers, created multiple fake identities, and used the fake identities to socially engineer a real maintainer into approving the code."*
- Detected via unusual data transfers; runs terminated within the hour. The pull request was closed after a separate human contributor flagged it.

An earlier version of this course had the order wrong — it said the sockpuppets were created *after* a maintainer refused. They were part of the persuasion campaign from the start. Small difference, different story, and the report says which.

Incident two — chased to the source

The story, as it circulates: agents used an internal package manager as a message board, coordinating undetected while an evaluation ran.

- The vendor's own disclosure documents a package-registry zero-day and a partner breach. It does **not** mention a message board at all.
- The message-board claim comes from a conference talk, reported in a magazine article. That is testimony relayed by journalism — no incident report anywhere in the chain.
- And the article could not be reached. Two independent routes: a direct fetch, blocked; a second tool reporting a 404 on the URL, no match on the headline, and the story absent from the author's own index.

So the chain ends like this: **no primary report, and no reachable secondary report either**. A widely repeated incident, followed to its source, arrives at nothing you can check. The duration usually quoted — "about a month" — appears in no source this course could open, so it is not stated here.

Self-propagating injection

The genuine worm case: an injected instruction that causes the output to carry the injection onward.

- Demonstrated against generative-AI ecosystems where one system's output becomes another's input. **The specific applications are not stated here** — the citation is confirmed, the paper is not yet read.
- It needs no software vulnerability. The propagation channel is the application working as designed.
- Peer-reviewed, and the preprint circulates under a **different title** — check which one you are citing.

Training-data poisoning

- Material placed in a training corpus, years before you touched anything, to change behaviour on a trigger.
- You cannot inspect it, test for it, or defend against it at your end. The corpus is not yours and the weights are frozen before you arrive.
- So it does not argue for a tool. It argues for the habit: verify outputs you rely on, whatever their provenance.

That is an uncomfortable slide, and leaving it out would be worse. A threat you cannot defend against still changes how much you trust the output.

What you can actually do

- **Separate instruction from data visibly** — the tagged structure from Chapter 3. It is not a guarantee; it raises the bar.
- **Constrain what the session can reach.** Read-only defaults, narrow permissions, and the diff review from Chapter 9.
- **Treat tool output as untrusted input, not as fact.** The protocol specification says the same of tool *descriptions* — they "should be considered untrusted, unless obtained from a trusted server."
- **Know what you pointed it at.** Most of your exposure is a choice about which document gets opened in a session that can write files.

[V] Model Context Protocol, 2025-06-18. *modelcontextprotocol.io* — *protocol specification*, version 2025-06-18.

[V] OWASP GenAI Security Project, 2026. *OWASP Top 10 for LLM Applications, 2026 edition*, LLM01:2026.

The demonstration

TODO(produce) — a README and a PDF containing instructions aimed at the agent, run against a session with file access. Assets live in `assets/injection-demo/`, isolated and never executed from the working machine.

Rehearse it in the week before delivery. Model behaviour on injection changes between versions, and a demo that quietly stopped working is worse than no demo. Have the run-sheet's failure branch ready.

If it does not trigger, that is not a wasted slot. "It was patched" and "it is fixed" are different claims, and the room should hear you make that distinction live.

Where this leaves us

- One channel, no envelope, and no parameterised-query equivalent to reach for.
- One documented incident, and one that is only testimony — and you now know which is which, which is the more useful thing to leave with.
- A threat class you cannot defend against, which is an argument for verification rather than for a product.

Thread — trust boundary. Introduced in Chapter 2 as trained behaviour, attacked here, and opened voluntarily in Chapter 12. Governed in Chapter 18.

Session 2 hands you tools that read your files and act on them. **You were shown the threat model first on purpose.**